LAPORAN PRATIKUM

STRUKTUR DATA

“Laporan Praktikum Pekan 4”

Disusun oleh:

Rifal maulana oskar

2511533024

Dosen Pengampu : Dr. Wahyudi, S.T, M.T

Asisten Praktikum : Sherly sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum ini dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum struktur data pada pertemuan keempat dengan materi dasar **queue**

Semoga laporan ini dapat memberikan pemahaman dasar mengenai penggunaan bahasa pemrograman Java, khususnya dalam mengenal struktur data.

Penulis menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Padang, 29 April 2026

Rifal maulana oskar

DAFTAR ISI

BAB I 4
1.1 Latar belakang 4
1.2 Tujuan 4
1.3 Manfaat 4
BAB II 5
2.1 Pengertian queue. 5
2.2 Operasi dasar queue. 5
2.3 Implementasi Queue Menggunakan Iterator (Iterasiqueue_2511533024.java) 6
2.4 Implementasi Queue Menggunakan Array (queueArray_2511533024.java) 6
2.6 Reverse Data Queue Menggunakan Stack (reverseData_2511533024.java) 7
BAB III 9
DAFTAR PUSTAKA 10

BAB I

PENDAHULUAN

1.1 Latar belakang

Struktur data merupakan fondasi penting dalam pengembangan perangkat lunak. Salah satu struktur data yang sering digunakan adalah *queue* (antrian) yang bekerja dengan prinsip *First In First Out* (FIFO). Konsep ini banyak diterapkan dalam sistem nyata, misalnya antrian pelanggan, penjadwalan proses, dan manajemen buffer data.

Mahasiswa perlu memahami implementasi *queue* secara mendalam, baik menggunakan *array* maupun *linked list*, serta operasi-operasi lanjutan seperti pembalikan (*reverse*) antrian. Oleh karena itu, dalam makalah ini akan dibahas implementasi *queue* dalam bahasa Java berdasarkan kode program yang telah dibuat oleh penulis dengan NIM 2511533024 (akhiran variabel _3024).

1.2 Tujuan

Tujuan dari penulisan makalah ini adalah:

- Menjelaskan teori dasar *queue* dan operasinya.
- Menganalisis kode program queueArray_2511533024, QueueArrayDriver_2511533024, queueLinkedlist _2511533024, Iterasiqueue_2511533024, dan reverseData_2511533024.
- Memberikan contoh implementasi lengkap yang memenuhi aturan penamaan dengan akhiran _3024.

.

1.3 Manfaat

- Sebagai bahan belajar struktur data khususnya *queue*.
- Sebagai referensi bagi mahasiswa lain yang ingin mengimplementasikan *queue* di Java.
- Memenuhi tugas mata kuliah Struktur Data dan Algoritma.

BAB II
PEMBAHASAN

2.1 Pengertian queue.

Queue atau antrian 5ar uk struktur data linear yang menyimpan elemen-elemen dalam urutan tertentu. Elemen baru ditambahkan di bagian belakang (*rear*) dan elemen yang dihapus berasal dari bagian depan (*front*). Prinsip ini disebut **FIFO** (First In First Out) – mirip dengan antrian orang di loket.

2.2 Operasi dasar queue.

Operasi	Deskripsi
enqueue	Menambahkan elemen ke belakang antrian
dequeue	Menghapus elemen dari depan antrian
front	5ar uke55lemen depan tanpa menghapus
rear	5ar uke55lemen belakang
isEmpty	Mengecek apakah antrian kosong
isFull	Mengecek apakah antrian penuh (khusus array)
display	Menampilkan seluruh isi antrian
reverse	Membalik urutan antrian

2.3 Implementasi Queue Menggunakan Iterator (Iterasiqueue_2511533024.java)

Pada file program **Iterasiqueue_2511533024.java**, program menggunakan interface Queue dari Java Collection Framework dengan implementasi LinkedList. Queue tersebut bertipe data String sehingga dapat menyimpan kata atau kalimat. Pada program ini, beberapa string dimasukkan ke dalam queue menggunakan method add(), yaitu kata "pratikum", "struktur", "data", "dan", dan "algoritma". Setelah itu, program menggunakan Iterator untuk membaca isi queue satu per satu. Iterator bekerja dengan cara memeriksa apakah masih ada elemen berikutnya menggunakan hasNext(), lalu mengambil elemen tersebut menggunakan next(). Selama iterator masih memiliki elemen, program akan terus mencetak isi queue. Hal ini membuktikan bahwa queue dapat ditampilkan tanpa harus menghapus datanya, karena iterator hanya membaca isi antrian. Program ini berguna untuk memahami bagaimana cara melakukan traversal atau penelusuran data queue menggunakan iterator.

2.4 Implementasi Queue Menggunakan Array (queueArray_2511533024.java)

Pada file **queueArray_2511533024.java**, queue diimplementasikan secara manual menggunakan array bertipe integer. Program ini memiliki 6ar uke6 front_3024 yang menunjuk posisi elemen terdepan, rear_3024 yang menunjuk posisi elemen paling belakang, size_3024 yang menyimpan jumlah elemen dalam queue, dan capacity yang menentukan kapasitas maksimal queue. Array dibuat berdasarkan kapasitas tersebut untuk menampung data. Konstruktor pada program digunakan untuk menginisialisasi kapasitas, mengatur front dan size menjadi nol, serta rear diatur menjadi capacity – 1. Hal ini dilakukan karena saat enqueue pertama dilakukan, rear akan bertambah satu dan mengarah ke indeks awal array.

Program menyediakan fungsi IsFull_3024 untuk memeriksa apakah queue sudah penuh, yaitu jika size sama dengan capacity. Program juga menyediakan IsEmpty_3024 untuk memeriksa apakah queue kosong, yaitu jika size bernilai nol. Pada method enqueue_3024, program akan menambahkan data 6ar uke belakang queue. Prosesnya dilakukan dengan menaikkan rear menggunakan rumus modulo agar queue menjadi circular queue atau antrian melingkar. Setelah rear bergeser, data dimasukkan ke array pada indeks rear, lalu size bertambah satu. Program juga menampilkan output bahwa item telah dimasukkan ke dalam queue.

Pada method dequeue_3024, program menghapus elemen dari posisi front. Jika queue kosong, program mengembalikan nilai Integer.MIN_VALUE sebagai penanda tidak ada data. Jika tidak kosong, item yang berada pada indeks front akan disimpan sementara, lalu front digeser menggunakan rumus modulo untuk mempertahankan konsep circular queue. Setelah itu size dikurangi satu dan item yang dihapus dikembalikan sebagai hasil method. Method front_3024 digunakan untuk mengambil elemen paling depan tanpa menghapus, sedangkan rear_3024 digunakan untuk mengambil elemen paling belakang.

Program juga memiliki method display_3024 untuk menampilkan isi queue. Program memeriksa kondisi jika front sama dengan rear maka dianggap queue kosong dan menampilkan pesan "Antrian kosong". Jika tidak kosong, program melakukan perulangan dari front sampai rear dan mencetak isi array satu per satu dengan format panah. Program ini menggambarkan bagaimana queue dapat dibuat tanpa menggunakan library bawaan Java.

2.4 Program Driver Queue Array (QueueArrayDriver_2511533024.java)

Pada file **QueueArrayDriver_2511533024.java**, program ini berfungsi sebagai driver atau penguji untuk kelas queueArray_2511533024. Program membuat objek queue dengan kapasitas 1000. Setelah itu dilakukan enqueue beberapa data yaitu 10, 20, 30, dan 40. Kemudian program menampilkan elemen paling depan dan paling belakang dari queue. Namun pada kode, pemanggilan front_3024 dan rear_3024 tidak menggunakan tanda kurung sehingga seharusnya ditulis sebagai front_3024() dan rear_3024(). Program lalu menampilkan isi queue menggunakan method display_3024.

Setelah itu, program melakukan operasi dequeue satu kali dan mencetak elemen yang dihapus. Kemudian program menampilkan kembali elemen front dan rear serta menampilkan isi queue setelah satu elemen dikeluarkan. Program ini menunjukkan proses kerja queue bahwa elemen pertama yang masuk (10) adalah elemen pertama yang keluar. Dengan driver ini, implementasi queue berbasis array dapat diuji sehingga dapat diketahui apakah enqueue, dequeue, dan display berjalan dengan benar.

2.4 Implementasi Queue Menggunakan LinkedList (queueLinkedlist_2511533024.java)

Pada file **queueLinkedlist_2511533024.java**, program menggunakan interface Queue dengan implementasi LinkedList. Queue ini bertipe Integer sehingga menyimpan data angka. Program melakukan perulangan dari i = 0 sampai i < 6, sehingga angka 0 hingga 5 dimasukkan ke dalam queue menggunakan method add(). Setelah itu program mencetak isi queue menggunakan System.out.println, yang akan menampilkan seluruh elemen dalam bentuk list.

Program kemudian melakukan penghapusan elemen paling depan menggunakan method remove(). Nilai yang dihapus disimpan dalam variabel hapus lalu ditampilkan. Hal ini menunjukkan bahwa queue selalu menghapus elemen paling depan terlebih dahulu. Setelah itu queue dicetak kembali untuk melihat perubahan. Selanjutnya program menggunakan method peek() untuk melihat elemen paling depan tanpa menghapusnya, lalu disimpan dalam variabel depan. Program juga menggunakan method size() untuk menghitung jumlah elemen yang tersisa dalam queue. Program ini menunjukkan bahwa penggunaan queue dengan LinkedList jauh lebih sederhana dibanding menggunakan array karena struktur datanya sudah disediakan oleh Java dan operasi dasar queue dapat langsung digunakan

2.6 Reverse Data Queue Menggunakan Stack (reverseData_2511533024.java)

Pada file **reverseData_2511533024.java**, program bertujuan untuk membalik urutan data yang ada di dalam queue. Program membuat queue menggunakan LinkedList dan memasukkan angka 1, 2, dan 3. Program menampilkan isi queue sebelum dilakukan reverse. Kemudian program membuat Stack bertipe Integer. Stack merupakan struktur data yang bekerja dengan prinsip LIFO (Last In First Out), yaitu data terakhir masuk akan keluar pertama.

Program kemudian menjalankan perulangan while selama queue tidak kosong. Pada proses ini, elemen queue dihapus menggunakan remove() lalu dimasukkan ke dalam stack menggunakan push(). Akibatnya seluruh isi queue berpindah ke stack. Setelah queue kosong, program melakukan perulangan kedua selama stack tidak kosong. Pada proses ini, elemen paling atas stack diambil menggunakan pop() lalu dimasukkan kembali ke queue menggunakan add(). Karena stack mengeluarkan data dengan urutan terbalik, maka ketika data dimasukkan kembali ke queue, urutannya menjadi terbalik dari sebelumnya. Akhirnya program mencetak isi queue setelah reverse. Program ini membuktikan bahwa kombinasi queue dan stack dapat digunakan untuk membalik urutan data secara efisien.

BAB III
KESIMPULAN

Berdasarkan pembahasan yang telah dijelaskan, dapat disimpulkan bahwa Queue merupakan struktur data yang menerapkan konsep FIFO (First In First Out) dimana elemen yang masuk lebih dulu akan keluar lebih dulu. Queue dapat diimplementasikan dalam Java menggunakan array maupun menggunakan LinkedList dari Java Collection Framework. Implementasi queue berbasis array memerlukan pengaturan variabel front, rear, size, serta mekanisme circular queue agar penggunaan array lebih efisien. Sementara itu, implementasi queue menggunakan LinkedList lebih mudah karena Java telah menyediakan fungsi enqueue, dequeue, peek, dan size secara otomatis.

Selain itu, queue juga dapat diiterasi menggunakan Iterator untuk membaca data tanpa menghapusnya. Proses reverse data queue juga dapat dilakukan dengan memanfaatkan struktur data Stack karena stack memiliki sifat LIFO sehingga dapat membalik urutan data secara otomatis. Dengan demikian, pemahaman implementasi queue sangat penting dalam dunia pemrograman karena queue sering digunakan dalam sistem komputer dan aplikasi nyata.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd Edition)*. MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java (6th Edition)*. Wiley.

Oracle. (2024). *Java Documentation - Queue Interface*. Oracle Java Documentation.

Oracle. (2024). *Java Documentation - LinkedList Class*. Oracle Java Documentation.

Kadir, Abdul. (2013). *Dasar Pemrograman Java*. Andi Publisher.